

CL2006 – Operating System lab (Spring 2026)

BS-4-A/B

Semester Project

**Concurrent Banking System with Scheduling,
Synchronization, IPC, and Memory Management**

Submission Guidelines

Original Work Required: Plagiarism will not be tolerated. Every student must submit their own work.

Grading Structure: The project carries both individual and group work marks.

Collaboration & Independence: Students are expected to cooperate with their group members but must also be able to present and explain the project individually.

Understanding the Task: Carefully reading and understanding the project questions is considered part of the assigned work.

Submission Guidelines:

- Upload your completed project on Google Classroom.
- Submit as a zip folder.
- The folder must be named according to the given instructions (e.g., "ConcurrentBankingSystem_GroupX").

Deadline: All submissions must be made by 08-05-2026. Late submissions will not be accepted. The examples given are just for the sake of understanding.

Project Guidelines:

Objectives

This project connects **Operating System concepts** with a practical banking simulation. Students will implement multithreading, scheduling, synchronization, IPC, deadlock handling, and memory management. Each module must produce measurable outputs (charts, logs, metrics) to demonstrate correctness.

Customer Types

- **Regular Customers** → Perform deposits/withdrawals. Example: Two regular customers deposit simultaneously; synchronization ensures balances remain correct.
- **Premium Customers** → Higher scheduling priority. Example: A premium customer's withdrawal executes before a regular customer's deposit in Priority Scheduling.
- **Loan Applicants** → Banker's Algorithm checks loan requests. Example: If granting a loan makes the system unsafe, the request is denied.
- **Corporate Clients** → Multiple threads per client. Example: A corporate client initiates payroll for 50 employees, spawning multiple threads.

National University of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

- **VIP Customers** → Highest scheduling priority. Example: A VIP customer's transaction always preempts others, testing fairness and starvation.

System Design Guidelines

1. Threading & Scheduling

Each customer request runs as a **thread/process**. The scheduler decides execution order using FCFS, Priority, or Round Robin.

- **Example (FCFS)**: Customer A arrives first, then B, then C → execution order is A → B → C.
- **Example (Priority)**: VIP (priority 5) executes before Premium (priority 3) and Regular (priority 1).
- **Example (Round Robin)**: Each customer gets a fixed time slice (e.g., 2 ms) → A executes for 2 ms, then B, then C, cycling until completion. Students must generate a **Gantt chart** and calculate metrics (waiting time, turnaround time).

2. Synchronization

Shared resources like account balances must be protected with **mutex locks**. **Semaphores** allow limited concurrent access.

- **Example (Mutex)**: Two customers withdraw from the same account → mutex ensures only one thread updates the balance at a time.
- **Example (Semaphore)**: A semaphore with value 2 allows two deposits at once, but blocks the third until one finishes. This prevents race conditions and ensures correctness in concurrent transactions.

3. Deadlock Handling

Loan requests are treated as resource allocation problems. Using **Banker's Algorithm**, the system checks if granting a loan keeps the system safe.

- **Example**: Bank has 10 units of credit. Customer A requests 7, Customer B requests 5. If A is granted first, B cannot be satisfied → unsafe state. Banker's Algorithm denies A's request to prevent deadlock. Students must demonstrate safe vs unsafe states with test cases.

4. Message Passing & IPC

Customer processes communicate with the bank server using **pipes or message queues**.

- **Example (Message Queue)**:
 - Customer process sends: Deposit 500 to Account #123.
 - Bank server reads from queue, processes deposit, and sends back: Transaction Successful.
- **Example (Producer-Consumer)**: Multiple customers (producers) send requests, while the bank server (consumer) processes them one by one. Students must log request/response flow to show IPC in action.

5. Memory Management

Each customer has a simulated **memory footprint** (account data, transaction history). The bank allocates frames, and page replacement algorithms decide which page to evict when memory is full.

- **Example (FIFO)**: Pages [A, B, C] in memory. New page D arrives → oldest page A is replaced.
- **Example (LRU)**: Pages [A, B, C] in memory. If A was least recently used, it is replaced when D arrives. Students must measure **page faults** and **hit ratios**, comparing FIFO vs LRU efficiency.

Deliverables

1. **Codebase** with modular design for each OS concept.
2. **Report** explaining implementation, test cases, charts, and logs.
3. **Demo** showing concurrency, scheduling differences, deadlock prevention, IPC communication, and memory paging simulation.